

P R E S E N T A T I O N

Assistance IA pour les Data Engineers dans Microsoft Fabric

Accélérer les développements Data Engineering & Data Science

Paul Peton

Architecte IA - Expertise

Agenda

A l'issue de la présentation, vos Data Engineers auront découvert...

1

Les fonctionnalités Copilot dans Fabric

Notebooks, Dataflows, pipelines, SQL...

2

Les principes du vibe coding

Nouvelle approche de développement de code par assistance IA

3

GitHub Copilot dans VSC

Installation, paramétrage, extensions

4

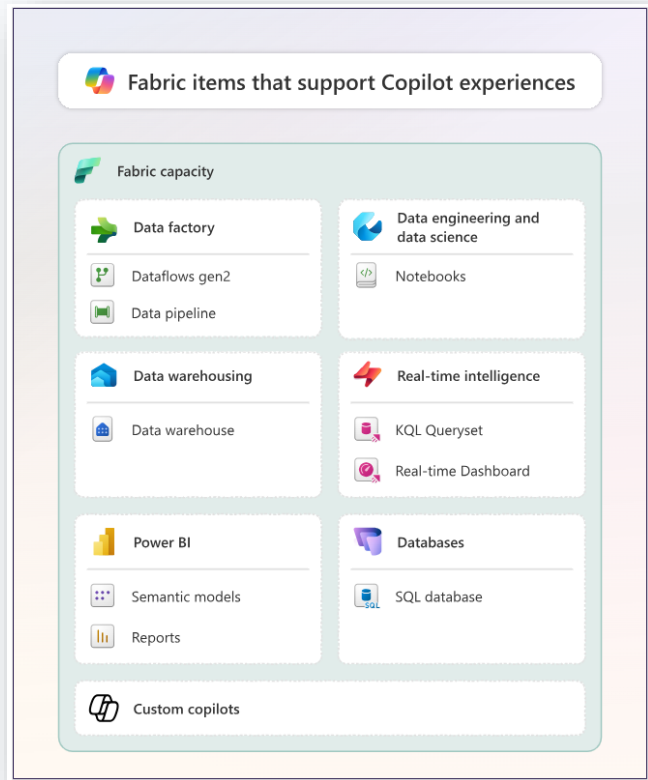
Les interactions entre GitHub Copilot et les items Fabric

Développement local, exécution à distance

1 Fonctionnalités Copilot dans Fabric

Chaque workload dispose de son Copilot

- Notebooks Data Engineering & Data Science
 - Workload Data Factory
 - Workload Data Warehouse
 - Workload Real-Time Intelligence
-
- Copilot for SQL database



Requêtes SQL dans le Data Warehouse

- Générer des requêtes SQL (preview).
- Suggérer des complétions de code SQL (preview).
- Corriger le code des requêtes SQL (preview).
- Expliquer le code des requêtes SQL (preview).

Copilot dans Fabric Data Warehouse

Copilot dans Microsoft Fabric Data Warehouse est un assistant IA conçu pour simplifier vos tâches d'entrepôt de données. Voici ce que Copilot peut faire :

- **Natural Language to SQL** : décrivez votre besoin en langage courant, et Copilot génère le T-SQL pour vous.
- **Code Completion** : dès que vous commencez à écrire du T-SQL, Copilot suggère des complétions pour écrire vos requêtes plus vite.
- **Quick Actions** : corrigez rapidement les erreurs de syntaxe SQL ou demandez à Copilot d'expliquer l'objectif d'une requête directement dans l'éditeur.
- **Intelligent Insights** : Copilot fournit des suggestions et des analyses utiles à partir du schéma et des métadonnées de votre entrepôt.

Vous pouvez utiliser Copilot via :

- Un chat pane pour poser des questions et générer des requêtes.
- La complétion de code inline pendant l'écriture du SQL.
- Des quick actions (comme Fix et Explain) dans le ruban pour corriger les erreurs ou comprendre le code.

Pour plus de détails dans [la documentation officielle](#).

Copilot Code Completion dans Fabric Data Warehouse (Preview)

Copilot Code Completion assiste les développeurs SQL en proposant des suggestions T-SQL intelligentes directement dans l'éditeur de requêtes Fabric Data Warehouse. Il s'appuie sur le schéma du Data Warehouse et le contexte de la requête en cours afin de générer des complétions pertinentes en temps réel.

Fonctionnalités principales

- Complétion automatique de requêtes T-SQL pendant la saisie.
- Suggestions contextuelles basées sur les tables, colonnes et objets du Data Warehouse.
- Génération de blocs de code complets, et non seulement de quelques mots.
- Support des commandes DDL, DML et DQL.
- Possibilité de générer du SQL à partir de commentaires rédigés en langage naturel.
- Proposition de plusieurs variantes de code pour une même demande.

Utilisation

Copilot affiche des suggestions directement dans l'éditeur sous forme de texte grisé (« ghost text »). Les utilisateurs peuvent accepter l'intégralité d'une proposition, n'en sélectionner qu'une partie ou choisir parmi plusieurs alternatives. Les commentaires en langage naturel peuvent également être utilisés pour décrire l'intention métier, Copilot générant ensuite la requête SQL correspondante.

 SQL query 2

+ Warehouses

- ds warehouse

▼ Schemas

▼ dbo

Tables

```
>  fabrikam_customer
```

>  fabrikam date

>  fabrikam geo

> fabrikam_people

```
>  fabrikam_po
```

```
>  fabrikam_product
```

> fabrikam productde

- >  fabrikam reseller

>  fabrikam sales

>  fabrikam suppl

>  Views

>  Functions

Q

Run

 Save as Save as view Copy query

 Create rule

 Explain query

 Preview Copilot uses AI. Mistakes can happen. Verify code suggestions before running them. [Review terms](#)

```
1 SELECT TOP (100) [CustomerID],
2     [Address],
3     [Age],
4     [BrandAffinity],
5     [City],
6     [Country],
7     [FullName],
8     [Generation],
9     [InterestAffinity1],
10    [InterestAffinity2],
11    [ResellerID],
12    [State],
13    [ZipCode]
14 FROM [ds_warehouse].[dbo].[fabrikam_customer]
```

Messages

Results

 Search

□ ×

	ABC CustomerID	ABC Address	123 Age	ABC BrandAffinity	ABC City	ABC Country	ABC FullName	ABC
1	c8a587f46d1f13daea...	1082 Schmidt Statio...	27	NULL	Miami	USA	Tara Considin	Ger
2	36e53f0e6037acb20...	610 Jerde Fort Unit 1...	19	NULL	Miami	USA	Angela Kih	Ger
3	151f363c591524187...	603 Greenfelder Vill...	21	NULL	Miami	USA	Dawn Gutkowski	Ger
4	208fe65e27caad895...	158 Douglas Overpa...	27	NULL	Miami	USA	Irma Ra	Ger
5	95c6e3fc8f5b07cd90...	841 Ondricka Height...	19	NULL	Miami	USA	Lisa Ric	Ger
6	bb67d11a4d84ad23...	410 Kub Neck Unit 6...	27	NULL	Miami	USA	Victoria Gleason	Ger
7	bf7da0b7b0dbdf806...	453 Emmerich Bay A...	20	NULL	Miami	USA	Molly Carroll	Ger

 [View prompts](#)

Ask me anything about this warehouse, or tell me how I can help.

0 / 500

AI-generated content can have mistakes. Make sure it's accurate and appropriate before using it. [Review terms](#)

Columns: 13 Rows: 100

Is this response helpful?

Ajoute une clause pour filtrer les zipCode sur 5 chiffres

This SQL query selects all rows from the table [dbo].[fabrikam_customer] where the ZipCode value is exactly five digits long. The query casts ZipCode to a string, pads it with zeros if needed, and then checks both the length and the numeric pattern. This approach ensures only records with five-digit numeric ZipCodes (as strings) are included and excludes NULLs.

SQL

```
-- Step 1: Select rows from [dbo].[fabrikam_customer] where [ZipCode] contains exactly 5 di
-- Step 2: Use a WHERE clause with LIKE and LEN to ensure [ZipCode] is 5 digits (and not NUL
-- Step 3: Since [ZipCode] is INT, convert it to VARCHAR for pattern matching and length chu

SELECT *
FROM [dbo].[fabrikam_customer]
WHERE
    -- Convert [ZipCode] to VARCHAR(5) for pattern matching
    LEN(RIGHT('00000' + CAST([ZipCode] AS VARCHAR(5)), 5)) = 5
    AND RIGHT('00000' + CAST([ZipCode] AS VARCHAR(5)), 5) LIKE '[0-9][0-9][0-9][0-9][0-9]'
    -- Exclude NULLs just in case
    AND [ZipCode] IS NOT NULL;
```

Next steps:

- Run the query and review the results to confirm only five-digit zip codes are returned.
- Consider saving this filter as a view if you'll need it often.
- Adjust the logic if zip codes can start with zero and should be preserved as text.

Do you want to apply this filter to another table or need more help with zip code formats?

Is this response helpful?

View prompts

Ask me anything about this warehouse, or tell me how I can help.

0 / 500

Fabric

ds_warehouse

Search

Analyze data with

Share

Home

Management

Help

Get data

New SQL query

SQL templates

Monitor

New semantic model

New API for GraphQL

Add to data agent

Download SQL database project

Open in

Copilot

Explorer

SQL query 2

Warehouses

ds_warehouse

Schemas

dbo

Tables

fabrikam_customer

fabrikam_date

fabrikam_geo

fabrikam_people

fabrikam_po

fabrikam_product

fabrikam_productdt

fabrikam_reseller

fabrikam_sales

fabrikam_supplier

Views

Functions

Stored Procedures

INFORMATION_SCHEMA

queryinsights

sys

Security

Queries

My queries

Run

Save as

Save as view

Copy query

Create rule

Explain query

Fix query errors

1

2

3

4

5

6

7

8

9

10

11

12

13

-- Step 1: Select rows from [dbo].[fabrikam_customer] where [ZipCode] contains exactly 5 digits.

-- Step 2: Use a WHERE clause with LIKE and LEN to ensure [ZipCode] is 5 digits (and not NULL).

-- Step 3: Since [ZipCode] is INT, convert it to VARCHAR for pattern matching and length check.

SELECT *

FROM [dbo].[fabrikam_customer]

WHERE

-- Convert [ZipCode] to VARCHAR(5) for pattern matching

LEN(RIGHT('00000' + CAST([ZipCode] AS VARCHAR(5)), 5)) = 5

AND RIGHT('00000' + CAST([ZipCode] AS VARCHAR(5)), 5) LIKE '[0-9][0-9][0-9][0-9][0-9]'

-- Exclude NULLs just in case

AND [ZipCode] IS NOT NULL;

Messages

Results

Save as table

Open in Excel

Explore this data (preview)

Limited to 10,000 rows

Copy

Search

	ABC CustomerID	ABC Address	ABC Age	ABC BrandAffinity	ABC City	ABC Country	ABC FullName	ABC Generation	ABC InterestAffinity1	ABC Interest
1	c8a58746d1f13dae...	1082 Schmidt Statio...	27	NULL	Miami	USA	Tara Considin	Gen-Z	Women's Empower...	Environme
2	36e53f0e6037acbf20...	610 Jerde Fort Unit ...	19	NULL	Miami	USA	Angela Kih	Gen-Z	Women's Empower...	Environme
3	151f963c591524167...	603 Greenfelder Vill...	21	NULL	Miami	USA	Dawn Gutkowski	Gen-Z	Women's Empower...	Environme
4	208fe65e27caad895...	158 Douglas Overpa...	27	NULL	Miami	USA	Irma Ra	Gen-Z	Women's Empower...	Environme
5	95c6e3fcd5b07cd90...	841 Ondricka Heigh...	19	NULL	Miami	USA	Lisa Ric	Gen-Z	Women's Empower...	Environme
6	bb67d1140848ad23...	410 Kub Neck Unit 6...	27	NULL	Miami	USA	Victoria Gleason	Gen-Z	Women's Empower...	Environme
7	bf7da0b7b0dbdf806...	453 Emmerich Bay A...	20	NULL	Miami	USA	Molly Carroll	Gen-Z	Women's Empower...	Environme
8	290c91733128d453a...	542 Botsford Vladuc...	20	NULL	Miami	USA	Brenda Nikolaus	Gen-Z	Women's Empower...	Environme
9	115bb9335eee1be...	941 Heidenreich Roa...	18	NULL	Miami	USA	Victoria Schneider	Gen-Z	Women's Empower...	Environme
10	233ecc21c7ba26590...	563 Stracke Bay,Mia...	21	NULL	Miami	USA	Leslie Wunsch	Gen-Z	Women's Empower...	Environme
11	7a63c377ffec47fb74...	610 Jerde Fort Unit ...	21	NULL	Miami	USA	Eva Schoen	Gen-Z	Women's Empower...	Environme
12	65754e3366a2c1020...	583 Denesik Station...	24	NULL	Miami	USA	Sheila Blanda	Gen-Z	Women's Empower...	Environme
13	94fc3729574959422...	492 White Road Unit...	20	NULL	Miami	USA	Jan Macejkovic	Gen-Z	Women's Empower...	Environme
14	9770878d98089336...	854 Feest Course Sul...	21	NULL	Miami	USA	Kristi O'Connerr	Gen-Z	Women's Empower...	Environme
15	b837c3083a52fdd0b...	910 Macejkovic Broo...	18	NULL	Miami	USA	Cassandra Wiegand	Gen-Z	Women's Empower...	Environme

Is this response helpful?

Ajoute une clause pour filtrer les ZipCode sur 5 chiffres

This SQL query selects all rows from the table [dbo].[fabrikam_customer] where the ZipCode value is exactly five digits long. The query casts ZipCode to a string, pads it with zeros if needed, and then checks both the length and the numeric pattern. This approach ensures only records with five-digit numeric ZipCodes (as strings) are included and excludes NULLs.

SQL

-- Step 1: Select rows from [dbo].[fabrikam_customer] where [ZipCode] contains exactly 5 digits.

-- Step 2: Use a WHERE clause with LIKE and LEN to ensure [ZipCode] is 5 digits (and not NULL).

-- Step 3: Since [ZipCode] is INT, convert it to VARCHAR for pattern matching and length check.

SELECT *

FROM [dbo].[fabrikam_customer]

WHERE

-- Convert [ZipCode] to VARCHAR(5) for pattern matching

LEN(RIGHT('00000' + CAST([ZipCode] AS VARCHAR(5)), 5)) = 5

AND RIGHT('00000' + CAST([ZipCode] AS VARCHAR(5)), 5) LIKE '[0-9][0-9][0-9][0-9][0-9]'

-- Exclude NULLs just in case

AND [ZipCode] IS NOT NULL;

Next steps:

Run the query and review the results to confirm only five-digit zip codes are returned.

Consider saving this filter as a view if you'll need it often

Ask me anything about this warehouse, or tell me how I can help.

0 / 500

AI-generated content can have mistakes. Make sure it's accurate and appropriate before using it. Review terms

AutoSave: On (Last saved: 1m ago)

Succeeded (5 sec 111 ms)

Copilot completions: On

Copilot completions: Ready

Columns: 13 Rows: 10000

Home

Workspaces

Copilot

OneLake catalog

Monitor

Real-Time

Workloads

datascience

...

Fabric

Fabric

ds_warehouse

Search

Analyze data with

Share

Home

Management

Help

Get data

New SQL query

SQL templates

Monitor

New semantic model

New API for GraphQL

Add to data agent

Download SQL database project

Open in

Copilot

Explorer

SQL query 3

Warehouses

ds_warehouse

Schemas

dbo

Tables

fabrikam_customer

fabrikam_date

fabrikam_geo

fabrikam_people

fabrikam_po

fabrikam_product

fabrikam_productdetails

fabrikam_reseller

fabrikam_sales

fabrikam_supplier

Views

Functions

Stored Procedures

INFORMATION_SCHEMA

queryinsights

sys

Security

Queries

My queries

Run

Save as

Save as view

Copy query

Create rule

Explain query

Fix query errors

1

2

3

4

5

6

7

8

SELECT TOP (100) [Date],

[Year],

[Month],

[Quarter],

[MonthName],

[MonthNameShort],

[StartOfMonth]

FROM [ds_warehouse].[dbo].[fabrikam_date]

Copilot completions: On

Copilot completions: Ready

ds_warehouse

Warehouse

About

Endorsement

Tags

SQL endpoint

Copilot

SQL audit logs

Restore points

Show Copilot completions

Automatically get inline Copilot SQL completions as you type.

Learn more

Default configuration

On

Messages

Results

Save as table

ABC Date

123 Year

1

2

3

4

5

6

7

8

9

10

11

12

13

14

2019-02-01 00:00:00,000

2019-02-02 00:00:00,000

2019-02-03 00:00:00,000

2019-02-04 00:00:00,000

2019-02-05 00:00:00,000

2019-02-06 00:00:00,000

2019-02-07 00:00:00,000

2019-02-08 00:00:00,000

2019-02-09 00:00:00,000

2019-02-10 00:00:00,000

2019-02-11 00:00:00,000

2019-02-12 00:00:00,000

2019-02-13 00:00:00,000

2019-02-14 00:00:00,000

2019

2019

2019

2019

2019

2019

2019

2019

2019

2019

2019

2019

2019

2019

2

2

2

2

2

2

2

2

2

2

2

2

2

2

1

1

1

1

1

1

1

1

1

1

1

1

1

1

February

February

February

February

February

February

February

February

February

February

February

February

February

February

Feb

Feb

Feb

Feb

Feb

Feb

Feb

Feb

Feb

Feb

Feb

Feb

Feb

Feb

2019-02-01 00:00:00,000

2019-02-01 00:00:00,000

2019-02-01 00:00:00,000

2019-02-01 00:00:00,000

2019-02-01 00:00:00,000

2019-02-01 00:00:00,000

2019-02-01 00:00:00,000

2019-02-01 00:00:00,000

2019-02-01 00:00:00,000

2019-02-01 00:00:00,000

2019-02-01 00:00:00,000

2019-02-01 00:00:00,000

2019-02-01 00:00:00,000

2019-02-01 00:00:00,000

AutoSave: On

Succeeded (4 sec 983 ms)

Copilot completions: On

Copilot completions: Ready

Columns: 7

Rows: 100

Fabric

ds_warehouse

Search

Home

Management

Help

Get data

New SQL query

SQL templates

Monitor

New semantic model

New API for GraphQL

Add to data agent

Download SQL database project

Open in

Copilot

Analyze data with

Share

Home

Workspaces

Copilot

OneLake catalog

Monitor

Real-Time

Workloads

datascience

...

Explorer

Warehouses

ds_warehouse

Schemas

dbo

Tables

fabrikam_customer

fabrikam_date

fabrikam_geo

fabrikam_people

fabrikam_po

fabrikam_product

fabrikam_productdetails

fabrikam_reseller

fabrikam_sales

fabrikam_supplier

Views

Functions

Stored Procedures

INFORMATION_SCHEMA

queryinsights

sys

Security

Queries

My queries

SQL query 3

Run

Save as

Save as view

Copy query

Create rule

Explain query

Fix query errors

Accept

Discard

```
1 -- Retrieve the first 100 rows from the date dimension table
2 -- This table provides various date components useful for time-based analyses
3 SELECT TOP (100)
4     [Date],           -- Full date value
5     [Year],           -- Year component of the date
6     [Month],          -- Month number (1-12) of the date
7     [Quarter],        -- Quarter (1-4) of the year for the date
8     [MonthName],      -- Full name of the month (e.g., January)
9     [MonthNameShort], -- Abbreviated month name (e.g., Jan)
10    [StartOfMonth]    -- Date representing the first day of the month
11 FROM
12    [ds_warehouse].[db].[fabrikam_date]
13
14
```

```
1 -- Retrieve the first 100 rows from the date dimension table
2 -- This table provides various date components useful for time-based analyses
3 SELECT TOP (100)
4     [Date],           -- Full date value
5     [Year],           -- Year component of the date
6     [Month],          -- Month number (1-12) of the date
7     [Quarter],        -- Quarter (1-4) of the year for the date
8     [MonthName],      -- Full name of the month (e.g., January)
9     [MonthNameShort], -- Abbreviated month name (e.g., Jan)
10    [StartOfMonth]    -- Date representing the first day of the month
11 FROM
12    [ds_warehouse].[dbo].[fabrikam_date]
13 -- Auto-Fix: Corrected database schema qualifier from [db] to [dbo] for the fabrikam_date table as
14
15
```

Messages

Copy messages

8:22:23 PM

Started executing on line 1
Msg 208, Level 16, State 1, Line 3
Invalid object name 'ds_warehouse.db.fabrikam_date'.

8:22:24 PM

Query execution time: 00:00:00.003 | Time to process and render results: 00:00:00.860 | Total duration: 00:00:00.863

AutoSave: On (Last saved: 1m ago)

Completed with errors (0 sec 863 ms)

Copilot completions: On

Copilot completions: Ready

Copilot pour Data Factory

- **Dataflows gen2**
 - [Générer une nouvelle requête](#)
- **Pipeline**
 - [Générer et exécuter un pipeline.](#)
 - [Résumer un pipeline.](#)
 - [Diagnostiquer les erreurs de pipeline.](#)

Génération de transformations avec Copilot

- Une fois les données chargées, Copilot peut générer des étapes de transformation à partir d'une instruction textuelle.
- Copilot ajoute alors une étape dans la requête sélectionnée. L'utilisateur peut vérifier le résultat dans :
 - la liste **Applied steps** ;
 - la barre de formule ;
 - l'aperçu des données.
- Copilot peut aussi produire des agrégations.

Exemple : `Count the total number of employees by City`

- Il peut appliquer des filtres plus avancés.

Exemple : `Only keep orders whose quantities are above the median value`

Les réponses de Copilot apparaissent sous forme de cartes dans le panneau Copilot, avec la possibilité d'annuler certaines actions tant que la requête n'a pas été modifiée manuellement.

Génération de requêtes de transformation par IA

- Copilot facilite la création de transformations sans écrire manuellement le code Power Query / M.
- L'utilisateur exprime son besoin en langage naturel.
- Copilot traduit cette demande en étapes de transformation.
- L'approche permet d'accélérer les tâches fréquentes :
 - filtrage ;
 - agrégation ;
 - nettoyage ;
 - enrichissement ;
 - préparation des données pour analyse.

Home

Workspaces

Copilot

OneLake catalog

Monitor

Real-Time

Workloads

datascience

...

Fabric

Fabric

ds_warehouse

DataflowByCopilot

Search

43

Power Query

Search (Alt + Q)

Get data

Connect to data source

OData

Other

[Learn more](#)

Connection settings

URL

https://services.odata.org/V4/Northwind/North...

Connection credentials

Connection

Create new connection

Connection name

https://services.odata.org/V4/Northwind/Northwind.svc/

Data gateway

(none)

Authentication kind

Anonymous

Privacy Level

None

☐ This connection can be used with on-premises data gateways and VNet data gateways.

Back

Cancel

Next

	1.2 OrderID	1.2 ProductID	1.2 UnitPrice	1.2 Quantity	1.2 Discount	1.2 Order	1.2 Product	1.2 Ship
1	10248	11	14	12	0	[Record]	[Record]	20
2	10248	42	9.8	10	0	[Record]	[Record]	20
3	10248	72	34.8	5	0	[Record]	[Record]	10
4	10249	14	18.6	9	0	[Record]	[Record]	20
5	10249	51	42.4	40	0	[Record]	[Record]	10
6	10250	41	7.7	10	0	[Record]	[Record]	20
7	10250	51	42.4	35	0.150000006	[Record]	[Record]	10
8	10250	65	16.8	15	0.150000006	[Record]	[Record]	20
9	10251	22	16.8	6	0.05000000075	[Record]	[Record]	20
10	10251	57	15.6	15	0.05000000075	[Record]	[Record]	20
11	10251	65	16.8	20	0	[Record]	[Record]	20
12	10252	20	64.8	40	0.05000000075	[Record]	[Record]	10
13	10252	33	2	25	0.05000000075	[Record]	[Record]	20
14	10252	60	27.2	40	0	[Record]	[Record]	20
15	10253	31	10	20	0	[Record]	[Record]	20
16	10253	39	14.4	42	0	[Record]	[Record]	20
17	10253	49	16	40	0	[Record]	[Record]	20
18	10254	24	3.6	15	0.150000006	[Record]	[Record]	20
19	10254	55	19.2	21	0.150000006	[Record]	[Record]	20
20	10254	74	8	21	0	[Record]	[Record]	20
21	10255	2	15.2	20	0	[Record]	[Record]	20
22	10255	16	13.9	35	0	[Record]	[Record]	20
23	10255	36	15.2	25	0	[Record]	[Record]	20
24	10255	59	44	30	0	[Record]	[Record]	10
25	10256	53	26.2	15	0	[Record]	[Record]	20
26	10256	77	10.4	12	0	[Record]	[Record]	20

AI-generated content can have mistakes. Make sure it's accurate and appropriate before using it. [Review terms](#)

- Categories
- Customers
- Invoices
- Order_Details**
- Orders
- Products

```
1 let
2     Source = OData.Feed("https://services.odata.org/V4/Northwind/Northwind.svc/", null, [Implementation = "2.0"]),
3     #"Navigation 1" = Source[Name = "Order_Details", Signature = "table"][Data],
4     // Adds a new column called "Ship" to the Order_Details table: if UnitPrice <= 30 then 20, otherwise 10, with data type Int64.
5     #"Add column" = Table.AddColumn(#"Navigation 1", "Ship", each if [UnitPrice] < 30 then 20 else 10, Int64.Type)
6 in
```

	1²₃ OrderID	1²₃ ProductID	1.2 UnitPrice	1²₃ Quantity	1.2 Discount	Order nr	Product nr	1²₃ Ship	
1	10248	11	14	12	0	[Record]	[Record]	20	
2	10248	42	9.8	10	0	[Record]	[Record]	20	
3	10248	72	34.8	5	0	[Record]	[Record]	10	
4	10249	14	18.6	9	0	[Record]	[Record]	20	
5	10249	51	42.4	40	0	[Record]	[Record]	10	
6	10250	41	7.7	10	0	[Record]	[Record]	20	
7	10250	51	42.4	35	0.150000006	[Record]	[Record]	10	
8	10250	65	16.8	15	0.150000006	[Record]	[Record]	20	
9	10251	22	16.8	6	0.05000000075	[Record]	[Record]	20	
10	10251	57	15.6	15	0.05000000075	[Record]	[Record]	20	
11	10251	65	16.8	20	0	[Record]	[Record]	20	
12	10252	20	64.8	40	0.05000000075	[Record]	[Record]	10	
13	10252	33	2	25	0.05000000075	[Record]	[Record]	20	
14	10252	60	27.2	40	0	[Record]	[Record]	20	
15	10253	31	10	20	0	[Record]	[Record]	20	
16	10253	39	14.4	42	0	[Record]	[Record]	20	
17	10253	49	16	40	0	[Record]	[Record]	20	
18	10254	24	3.6	15	0.150000006	[Record]	[Record]	20	
19	10254	55	19.2	21	0.150000006	[Record]	[Record]	20	
20	10254	74	8	21	0	[Record]	[Record]	20	
21	10255	2	15.2	20	0	[Record]	[Record]	20	
22	10255	16	13.9	35	0	[Record]	[Record]	20	

Completed (0.99 s) Columns: 8 Rows: 99+

→ Add default destination... Not run

Query settings

▼ Properties

Name
Order_Details

- Applied steps

Source Naviga... Add co...

- ▼ Data destination

No data destination

 Copilot

add comments in the query

7:14 PM

Here's an explanation for **Add column** in **Order_Details**: Adds a new column called "Ship" to the Order_Details table: if UnitPrice < 30 then 20, otherwise 10, with data type Int64.

7:15 PM

Add to query

7:15 PM

Ok, I updated the description property of your step.

Describe what you'd like to do

AI-generated content can have mistakes. Make sure it's accurate and appropriate before using it. [Review terms](#)

Copilot pour Data Engineering et Data Science

- [Copilot chat pane](#) dans un notebook, avec génération, refactorisation et validation de code à l'échelle du notebook.
- [Générer du code ou du markdown dans les workflows de notebook \(preview\)](#).
- [Ajouter des commentaires, corriger des erreurs ou optimiser le code du notebook \(preview\)](#).
- [Analyser et visualiser les données](#).
- [Fix with Copilot \(preview\)](#) : résumés d'erreurs, analyse des causes racines et corrections de code soumises à validation, depuis le chat pane ou le Copilot in-cell.

Copilot Chat Pane pour Data Engineering & Data Science (Preview)

Copilot dans les notebooks Fabric agit comme un assistant IA conversationnel capable d'accompagner les utilisateurs tout au long du cycle de développement, d'analyse et d'optimisation de notebooks. Il exploite automatiquement le contexte du notebook (Lakehouse, tables, schémas, code existant et environnement d'exécution) pour fournir une assistance pertinente.

Fonctionnalités principales

- Interaction conversationnelle multi-étapes via un panneau de chat intégré.
- Génération automatique de code PySpark, Spark SQL et Python.
- Exploration, profilage et validation des données.
- Explication, correction, documentation et optimisation du code existant.
- Analyse des erreurs et assistance au dépannage des notebooks.
- Gestion de modifications avec visualisation des différences (diff view) avant validation.
- Historique des conversations conservé entre les sessions.

Fonctionnalités avancées

- Génération de code directement dans les cellules du notebook.
- Commandes rapides :
 - **/explain** : expliquer le code.
 - **/fix** : corriger les erreurs.
 - **/comments** : documenter le code.
 - **/optimize** : optimiser les performances et la qualité du code.

Aide à l'optimisation et au diagnostic

- Recommandations sur les jointures, les shuffles et les performances Spark.
- Identification de problèmes de qualité des données.
- Diagnostic automatique des échecs d'exécution.
- Analyse des logs Spark et proposition de correctifs.
- Suggestions de corrections applicables après validation de l'utilisateur.

Prompter en contexte data

Un bon prompt donne le contexte, pas seulement la demande : données, intention, contraintes.

PROMPT VAGUE

« écris du code PySpark pour agréger les ventes »

PROMPT CONTEXTUALISÉ

« En PySpark, sur `df_sales (date, montant, client_id)` : CA client, montants null $\rightarrow$ 0, trié décroissant »



Donner les données

Nom du DataFrame, colonnes, types, volumétrie.



Le commentaire d'abord

Écrire l'intention en commentaire, laisser compléter.



Découper

Une cellule = une étape logique nommée.

...

Fabric

ds_warehouse

NotebookDEByCopilot

Saved

Search

Home

Workspaces

Copilot

OneLake catalog

Monitor

Real-Time

Workloads

datascience

...

Fabric

HomeEditAI toolsRunView

Run allConnectPySpark (Python)EnvironmentWorkspace defaultData WranglerCopilot

Explorer

Data itemsResourcesConnections

No data sources added

Add data items

1# Welcome to your new notebook

2# Type here in the cell editor to add code!

3

PySpark (Python)

Copilot

Hello Paul!

Build autonomously with agent mode

Understand & Document

Summarize this notebook and add Markdown.

Add Markdown to explain my functions.

Show other notebooks related to this one.

Explore & Validate Data

Clean & Prepare Data

Clean my dataset, fix missing values and duplicates.

Normalize or standardize a column for consistency.

Analyze, Optimize & Convert

Show top metrics or trends in my data.

Convert this Python cell into SQL.

Agent

Let agent take action on your behalf.

Ask

Ask questions and get suggestions.

AI-generated content may be incorrect

Selected Cell 1 of 1 cells

Ajoute la source ds_lakehouse

NotebookDEByCopilot

Copilot ⓘ

✓ Use get_notebook_attached_lakehouse

Run tool get_artifact_id_by_name?

Allow



Skip

NotebookDEByCopilot •... +

Ask questions and work with this notebook

+ Agent ▾

AI-generated content may be incorrect

Selected Cell 1 of 1 cells

Approval settings ⓘ

✓ Ask for approval

Don't ask for approval

Copilot ⓘ

✓ Use get_notebook_attached_lakehouse

✓ Use get_artifact_id_by_name

✓ Set the default lakehouse.

La source ds_lakehouse a été ajoutée et définie comme lakehouse par défaut pour ce notebook.



NotebookDEByCopilot •... +

Ask questions and work with this notebook

+ Agent ▾

AI-generated content may be incorrect

Selected Cell 1 of 1 cells

...

Fabric

ds_warehouse

NotebookDEByCopilot

Saved

Search

Home

Edit

AI tools

Run

View

Cancel all

Standard session

PySpark (Python)

Environment

Workspace default

Data Wrangler

Copilot

Comments

History

Develop

Share

Session started in 5 sec

Explorer

Data items

Resources

Connections

+ Add data items

OneLake

ds_lakehouse

Tables

DA_dwh_evaluation_output

DA_dwh_evaluation_output_steps

DA_semantic_model_evaluation_output

DA_semantic_model_evaluation_output_steps

fabrikam_customer

fabrikam_date

fabrikam_geo

fabrikam_people

fabrikam_po

fabrikam_product

fabrikam_productdetails

fabrikam_reseller

fabrikam_sales

fabrikam_supplier

flights

germancredit

Files

1

Welcome to your new notebook

2

Type here in the cell editor to add code!

3

PySpark (Python)

1

Chargement des tables sales et customer dans des DataFrames

2

Table sales

4

df_sales = spark.read.table("ds_lakehouse.dbo.sales")

5

Table customer

7

df_customer = spark.read.table("ds_lakehouse.dbo.customer")

8

Aperçu des données

10

display(df_sales.limit(5))

11

display(df_customer.limit(5))

PySpark (Python)

8 sec

Keep all

Undo all

1 of 1

↑

↓

Copilot

Preview

Use get_notebook_attached_lakehouse

Use get_artifact_id_by_name

Set the default lakehouse.

La source ds_lakehouse a été ajoutée et définie comme lakehouse par défaut pour ce notebook.

Charge le contenu des tables sales et customer dans deux dataframes

NotebookDEByCopilot

Copilot

Je vais ajouter un nouveau bloc de code pour charger les tables sales et customer du lakehouse ds_lakehouse dans deux DataFrames, puis exécuter ce bloc pour vérifier que le chargement fonctionne.

Cell added: Cell #2

Use run_notebook_cells

1 cell changed +11-0

NotebookDEByCopilot

Ask questions and work with this notebook

+ Agent

AI-generated content may be incorrect

Selected Cell 2 of 2 cells

Home

Workspaces

Copilot

OneLake catalog

Monitor

Real-Time

Workloads

datascience

...

Fabric

Fabric

ds_warehouse

NotebookDEByCopilot

Saved

Search

Home

Edit

AI tools

Run

View

Run all

Standard session

PySpark (Python)

Environment

Workspace default

Data Wrangler

Copilot

Comments

History

Develop

Share

Explorer

Data items

Resources

Connections

Add data items

OneLake

ds_lakehouse

Tables

DA_dwh_evaluation_output

DA_dwh_evaluation_output_steps

DA_semantic_model_evaluation_output

DA_semantic_model_evaluation_output_steps

fabrikam_customer

fabrikam_date

fabrikam_geo

fabrikam_people

fabrikam_po

fabrikam_product

fabrikam_productdetails

fabrikam_reseller

fabrikam_sales

fabrikam_supplier

flights

germancredit

Files

1

2

3

4

5

6

7

8

9

10

11

12

13

14

15

16

17

18

19

20

21

22

23

24

25

26

27

28

29

30

31

32

33

34

35

36

37

38

39

40

41

42

43

44

45

46

47

48

49

50

51

52

53

54

55

56

57

58

59

60

61

62

63

64

65

66

67

68

69

70

71

72

73

74

75

76

77

78

79

80

81

82

83

84

85

86

87

88

89

90

91

92

93

94

95

96

97

98

99

100

101

102

103

104

105

106

107

108

109

110

111

112

113

114

115

116

117

118

119

120

121

122

123

124

125

126

127

128

129

130

131

132

133

134

135

136

137

138

139

140

141

142

143

144

145

146

147

148

149

150

151

152

153

154

155

156

157

158

159

160

161

162

163

164

165

166

167

168

169

170

171

172

173

174

175

176

177

178

179

180

181

182

183

184

185

186

187

188

189

190

191

192

193

194

195

196

197

198

199

200

201

202

203

204

205

206

207

208

209

210

211

212

213

214

215

216

217

218

219

220

221

222

223

224

225

226

227

228

229

230

231

232

233

234

235

236

237

238

239

240

241

242

243

244

245

246

247

248

249

250

251

252

253

254

255

256

257

258

259

260

261

262

263

264

265

266

267

268

269

270

271

272

273

274

275

276

277

278

279

280

281

282

283

284

285

286

287

288

289

290

291

292

293

294

295

296

297

298

299

300

301

302

303

304

305

306

307

308

309

310

311

312

313

314

315

316

317

318

319

320

321

322

323

324

325

326

327

328

329

330

331

332

333

334

335

336

337

338

339

340

341

342

343

344

345

346

347

348

349

350

351

352

353

354

355

356

357

358

359

360

361

362

363

364

365

366

367

368

369

370

371

372

373

374

375

376

377

378

379

380

381

382

383

384

385

386

387

388

389

390

391

392

393

394

395

396

397

398

399

400

401

402

403

404

405

406

407

408

409

410

411

412

413

414

415

416

417

418

419

420

421

422

423

424

425

426

427

428

429

430

431

432

433

434

435

436

437

438

439

440

441

442

443

444

445

446

447

448

449

450

451

452

453

454

455

456

457

458

459

460

461

462

463

464

465

466

467

468

469

470

471

472

473

474

475

476

477

478

479

480

481

482

483

484

485

486

487

488

489

490

491

492

493

494

495

496

497

498

499

500

501

502

503

504

505

506

507

508

509

510

511

512

513

514

515

516

517

518

519

520

521

522

523

524

525

526

527

528

529

530

531

532

533

534

535

536

537

538

539

540

541

542

543

544

545

546

547

548

549

550

551

552

553

554

555

556

557

558

559

560

561

562

563

564

565

566

567

568

569

570

571

572

573

574

575

576

577

578

579

580

581

582

583

584

585

586

587

588

589

590

591

592

593

594

595

596

597

598

599

600

601

602

603

604

605

606

607

608

609

610

611

612

613

614

615

616

617

618

619

620

621

622

623

624

625

626

627

628

629

630

631

632

633

634

635

636

637

638

639

640

641

642

643

644

645

646

647

648

649

650

651

652

653

654

655

656

657

658

659

660

661

662

663

664

665

666

667

668

669

670

671

672

673

674

675

676

677

678

679

680

681

682

683

684

685

686

687

688

689

690

691

692

693

694

695

696

697

698

699

700

701

702

703

704

705

706

707

708

709

710

711

712

713

714

715

716

717

718

719

720

721

722

723

724

725

726

727

728

729

730

731

732

733

734

735

736

737

738

739

740

741

742

743

744

745

746

747

748

749

750

751

752

753

754

755

756

757

758

759

760

761

762

763

764

765

766

767

768

769

770

771

772

773

774

775

776

777

778

779

780

781

782

783

784

785

786

787

788

789

790

791

792

793

794

795

796

797

798

799

800

801

802

803

804

805

806

807

808

809

810

811

812

813

814

815

816

817

818

819

820

821

822

823

824

825

826

827

828

829

830

831

832

833

834

835

836

837

838

839

840

841

842

843

844

845

846

847

848

849

850

851

852

853

854

855

856

857

858

859

860

861

862

863

864

865

866

867

868

869

870

871

872

873

874

875

876

877

878

879

880

881

882

883

884

885

886

887

888

889

890

891

892

893

894

895

896

897

898

899

900

901

902

903

904

905

906

907

908

909

910

911

912

913

914

915

916

917

918

919

920

921

922

923

924

925

926

927

928

929

930

931

932

933

934

935

936

937

938

939

940

941

942

943

944

945

946

947

948

949

950

951

952

953

954

955

956

957

958

959

960

961

962

963

964

965

966

967

968

969

970

971

972

973

974

975

976

977

978

979

980

981

982

983

984

985

986

987

988

989

990

991

992

993

994

995

996

997

998

999

1000

Keep

Undo

2 sec - Command executed in 2 sec 509 ms by Paul PETON on 06/07/2026 21:05:28

PySpark (Python)

Spark jobs (2 of 2 succeeded)

Resources

Schema de fabrikam_sales

root

-- InvoiceLineID: integer (nullable = true)

-- InvoiceID: integer (nullable = true)

-- StockItemID: integer (nullable = true)

-- Quantity: integer (nullable = true)

-- UnitPrice: double (nullable = true)

-- TaxRate: integer (nullable = true)

-- TaxAmount: double (nullable = true)

-- ResellerID: integer (nullable = true)

-- SalespersonPersonID: integer (nullable = true)

-- InvoiceDate: timestamp (nullable = true)

-- LineProfit: double (nullable = true)

-- ExtendedPrice: double (nullable = true)

-- SalesAmount: double (nullable = true)

Aperçu de fabrikam_sales

Table

New chart

13 columns, 5 rows

Download

Filter by keyword

Table view

123 InvoiceLineID

123 InvoiceID

123 StockItemID

123 Quantity

12 UnitPrice

123 TaxRate

12 TaxAmount

123 Resel

1

201128

62043

222

72

8.55

10

61.56

539

Keep all

Undo all

1 of 2

↑

↓

Copilot

Preview

profile tables to check columns names

NotebookDEByCopilot

Copilot

Use lakehouse_list_schemas

Use get_notebook_attached_lakehouse

Use lakehouse_list_tables

Use run_notebook_cells

Cell added: Cell #5

Use run_notebook_cells

I've added and executed a profiling cell to inspect the tables' schemas and sample rows.

Here is a summary of the column names:

Table fabrikam_sales

InvoiceLineID (int)

InvoiceID (int)

StockItemID (int)

Quantity (int)

UnitPrice (double)

TaxRate (int)

TaxAmount (double)

ResellerID (int)

SalespersonPersonID (int)

InvoiceDate (timestamp)

2 cells changed +24 -6

Keep

Undo

NotebookDEByCopilot

Ask questions and work with this notebook

+ Agent

AI-generated content may be incorrect

Selected Cell 5 of 9 cells

Explorer

 Data items Resources Connections

+ Add data items

OneLake

ds_lakehouse

▼ Tables

```
> DA_dwh_evaluation_output
```

```
> DA_dwh_evaluation_output_steps
```

```
> DA_semantic_model_evaluation_output
```

>  DA_semantic_model_evaluation_output_steps

> fabrikam_customer

>  fabrikam_date

> fabrikam_geo

>  fabrikam_people

> fabrikam_po

```
> fabrikam_product
```

> fabrikam_productdetails

> fabrikam_reseller

>  fabrikam_sales

> fabrikam_supplier

> flights

>  germancredit

> Files

```

2 # Création d'un Dataframe avec le montant total des ventes par client
3
4 from pyspark.sql import functions as F
5
6 # Inspection rapide des schémas pour utiliser les bons noms de colonnes
7 print("df_sales columns:", df_sales.columns)
8 print("df_customer columns:", df_customer.columns)
9
10 # 1. Jointure entre les ventes et les clients
11 # - Adapter "customer_id" et "sales_amount" si les noms de colonnes diffèrent dans vos tables.
12 # La table des ventes n'a pas de colonne customer_id mais une colonne ResellerID
13 # La table des clients a une colonne CustomerID et aussi ResellerID.
14 # On va supposer que le lien logique se fait via ResellerID.
15
16 # Jointure sur ResellerID
17 # -> s.ResellerID (ventes) avec c.ResellerID (clients)
18 df_sales_customer = (
19     df_sales.alias("s")
20     .join(
21         df_customer.alias("c"),
22         F.col("s.customer_id") == F.col("c.customer_id"),
23         F.col("s.ResellerID") == F.col("c.ResellerID"), # utiliser les colonnes réellement présentes
24         how="left"
25     )
26 )
27
28 # 2. Agrégation : montant total des ventes par client
29 # On agrège par l'identifiant client réel: CustomerID
30
31 # Vérifier qu'on a bien CustomerID dans le résultat de la jointure
32 # (il vient de df_customer alias "c")
33
34 # Agrégation par CustomerID
35 from pyspark.sql.functions import col
36
37 if "CustomerID" in df_sales_customer.columns:
38     df_sales_per_customer = (
39         df_sales_customer
40         .groupBy("c.customer_id")
41         .groupBy(col("c.CustomerID")) # utiliser la colonne CustomerID du côté client
42         .agg(
43             F.sum(F.col("s.SalesAmount")).alias("total_sales_amount")
44             F.sum(col("s.SalesAmount")).alias("total_sales_amount") # utiliser SalesAmount
45         )
46     )
47 
```

1 of 1

Session ready AutoSave: On Copilot completions: Off 1

 Copilot ⓘ

- ✓ Use `spark_get_diagnostics_context_for_current_cell`
- ✓ Code cell updated successfully.

Error Summary

Error type: AnalysisException (unresolved column in join condition)

Root cause: The original code tried to join on `s.customer_id` and `c.customer_id`, but in your real tables the columns are named `ResellerID` (in `fabrikam_sales`) and `CustomerID / ResellerID` (in `fabrikam_customer`). Spark couldn't find `customer_id` in `df_sales`, causing the failure.

Suggested Code Fix (already applied to the cell)

Python

```
# Création d'un DataFrame avec le montant total des ventes
from pyspark.sql import functions as F

# Inspection rapide des schémas pour utiliser les fonctions
print("df sales columns:", df_sales.columns)
```

1 cell changed +37 -8

NotebookDEByCopilot • ... +

Ask questions and work with this notebook

+ Agent ▾

AI-generated content may be incorrect

Selected Cell 4 of 4 cells

Home

Workspaces

Copilot

OneLake catalog

Monitor

Real-Time

Workloads

datascience

...

Fabric

Fabric

ds_warehouse

NotebookDEByCopilot

Search

Home

Edit

AI tools

Run

View

Run all

Standard session

PySpark (Python)

Environment

Workspace default

Data Wrangler

Copilot

Comments

History

Develop

Share

Explorer

Data items

Resources

Connections

+ Add data items

OneLake

ds_lakehouse

Tables

DA_dwh_evaluation_output

DA_dwh_evaluation_output_steps

DA_semantic_model_evaluation_output

DA_semantic_model_evaluation_output_steps

fabrikam_customer

fabrikam_date

fabrikam_geo

fabrikam_people

fabrikam_po

fabrikam_product

fabrikam_productdetails

fabrikam_reseller

fabrikam_sales

fabrikam_supplier

flights

germancredit

Files

35

36

37

38

39

40

41

42

43

44

45

alpha=0.7

plt.xlabel("Montant total des ventes (total_sales_amount)")

plt.ylabel("Jitter (pour la visualisation)")

plt.title("Visualisation des 3 clusters en fonction du montant total des ventes")

plt.legend()

plt.grid(True)

plt.tight_layout()

plt.show()

[7]

10 sec - Command executed in 10 sec 667 ms by Paul PETON on 06/07/2026 20:57:05

PySpark (Python)

Spark jobs (4 of 4 succeeded)

Resources

Visualisation des 3 clusters en fonction du montant total des ventes

Cluster 0

Cluster 1

Cluster 2

Jitter (pour la visualisation)

Montant total des ventes (total_sales_amount)

Session ready

AutoSave: On

Copilot completions: Off

Selected Cell 6 of 6 cells

Home

Workspaces

Copilot

OneLake catalog

Monitor

Real-Time

Workloads

datascience

...

Fabric

Fabric

ds_warehouse

NotebookDEByCopilot

Saved

Search

HomeEditAI toolsRunView

Run all

Standard session

PySpark (Python)

Environment

Workspace default

Data Wrangler

Copilot

Explorer

Data itemsResourcesConnections

Add data items

OneLake

ds_lakehouse

Tables

DA_dwh_evaluation_output

DA_dwh_evaluation_output_steps

DA_semantic_model_evaluation_output

DA_semantic_model_evaluation_output_steps

fabrikam_customer

fabrikam_date

fabrikam_geo

fabrikam_people

fabrikam_po

fabrikam_product

fabrikam_productdetails

fabrikam_reseller

fabrikam_sales

fabrikam_supplier

flights

germancredit

Files

```
7
8 import numpy as np
9 import matplotlib.pyplot as plt
10
11 # 1. Passage des données Spark -> Pandas pour la visualisation
12 # On part du DataFrame df_clustered_simple qui contient déjà la colonne "cluster"
13 # et la colonne "total_sales_amount" encapsulée dans "features".
14
15 from pyspark.sql.functions import col
16
17 # Récupération des valeurs sous forme de Pandas DataFrame
18 pdf = df_clustered_simple.select("total_sales_amount", "cluster").toPandas()
19
20 # 2. Création d'une petite composante "jitter" pour séparer les points sur l'axe Y
21 # Cela permet d'avoir un nuage de points lisible en 2D,
22 # même si toute l'information est portée par total_sales_amount (axe X).
23 np.random.seed(42)
24 pdf["jitter"] = np.random.normal(loc=0.0, scale=0.1, size=len(pdf))
25
26 # 3. Nuage de points : total_sales_amount en abscisse, jitter en ordonnée, coloré par cluster
27 plt.figure(figsize=(8, 6))
28
29 for cluster_id in sorted(pdf["cluster"].unique()):
30     subset = pdf[pdf["cluster"] == cluster_id]
31     plt.scatter(
32         subset["total_sales_amount"],
33         subset["jitter"],
34         label=f"Cluster {cluster_id}",
35         alpha=0.7
36     )
37
38 plt.xlabel("Montant total des ventes (total_sales_amount)")
39 plt.ylabel("Jitter (pour la visualisation)")
40 plt.title("Visualisation des 3 clusters en fonction du montant total des ventes")
41 plt.legend()
42 plt.grid(True)
43 plt.tight_layout()
44 plt.show()
45
```

10 sec - Command executed in 10 sec 667 ms by Paul PETON on 06/07/2026 20:57:05

PySpark (Python)

Spark jobs (4 of 4 succeeded)Resources

Visualisation des 3 clusters en fonction du montant total des ventes

Cluster 0

Cluster 1

Cluster 2

Copilot Preview

/comments

NotebookDEByCopilot

Copilot

Here are the key comments on what was done and why:

1. Root cause of the error

You tried to apply **PCA with k=2** on a feature vector of size 1 (total_sales_amount only).

Spark ML requires: *dimension of features* $\geq k$, so $1 < 2 \rightarrow$ `IllegalArgumentException`.

2. What I changed in the notebook

In `Cell #8`, I **removed the PCA step** and replaced it with:

- Extraction of total_sales_amount and cluster from df_clustered_simple into a Pandas DataFrame.
- Creation of a small random jitter column on the Y-axis, just for visualization.
- A scatter plot: **X = total_sales_amount**, **Y = jitter**, color = cluster.

3. Why this is a correct fix

Your clustering model is 1D (only total_sales_amount), so the cluster structure is fully described on a **single axis**.

Instead of forcing an artificial 2D reduction, we show a clear 1D distribution, just spread vertically to avoid overlap.

The K-means part in the previous cell remains unchanged and valid.

NotebookDEByCopilot

Ask questions and work with this notebook

Agent

AI-generated content may be incorrect

Selected Cell 8 of 8 cells

Home

Workspaces

Copilot

OneLake catalog

Monitor

Real-Time

Workloads

data science

...

Fabric

Fabric

ds_warehouse

NotebookDEByCopilotSaved

Search

HomeEditAI toolsRunView

Run all

Standard session

PySpark (Python)

Environment

Workspace default

Data Wrangler

Copilot

Explorer

Data ItemsResourcesConnections

+ Add data items

OneLake

ds_lakehouse

Tables

DA_dwh_evaluation_output

DA_dwh_evaluation_output_steps

DA_semantic_model_evaluation_output

DA_semantic_model_evaluation_output_steps

fabrikam_customer

fabrikam_date

fabrikam_geo

fabrikam_people

fabrikam_po

fabrikam_product

fabrikam_productdetails

fabrikam_reseller

fabrikam_sales

fabrikam_supplier

flights

germancredit

Files

```
7
8 import numpy as np
9 import matplotlib.pyplot as plt
10
11 # 1. Passage des données Spark -> Pandas pour la visualisation
12 # On part du DataFrame df_clustered_simple qui contient déjà la colonne "cluster"
13 # et la colonne "total_sales_amount" encapsulée dans "features".
14
15 from pyspark.sql.functions import col
16
17 # Récupération des valeurs sous forme de Pandas DataFrame
18 pdf = df_clustered_simple.select("total_sales_amount", "cluster").toPandas()
19
20 # 2. Création d'une petite composante "jitter" pour séparer les points sur l'axe Y
21 # Cela permet d'avoir un nuage de points lisible en 2D,
22 # même si toute l'information est portée par total_sales_amount (axe X).
23 np.random.seed(42)
24 pdf["jitter"] = np.random.normal(loc=0.0, scale=0.1, size=len(pdf))
25
26 # 3. Nuage de points : total_sales_amount en abscisse, jitter en ordonnée, coloré par cluster
27 plt.figure(figsize=(8, 6))
28
29 for cluster_id in sorted(pdf["cluster"].unique()):
30     subset = pdf[pdf["cluster"] == cluster_id]
31     plt.scatter(
32         subset["total_sales_amount"],
33         subset["jitter"],
34         label=f"Cluster {cluster_id}",
35         alpha=0.7
36     )
37
38 plt.xlabel("Montant total des ventes (total_sales_amount)")
39 plt.ylabel("Jitter (pour la visualisation)")
40 plt.title("Visualisation des 3 clusters en fonction du montant total des ventes")
41 plt.legend()
42 plt.grid(True)
43 plt.tight_layout()
44 plt.show()
45
```

✓ 10 sec - Command executed in 10 sec 667 ms by Paul PETON on 06/07/2026 20:57:05

Spark jobs (4 of 4 succeeded)Resources

Visualisation des 3 clusters en fonction du montant total des ventes

Cluster 0

Cluster 1

Cluster 2

Copilot Preview

/explain

NotebookDEByCopilot

Copilot

Here's what is going on, step by step, and why we changed the code the way we did.

1. Contexte : ce que fait votre notebook

1. Construction de la One Big Table (OBT)

- Vous partez des tables fabrikam_sales et fabrikam_customer.
- Vous joignez les ventes aux clients via ResellerID et obtenez un DataFrame enrichi df_sales_customer.
- Vous agrégez ensuite au grain client : df_sales_per_customer = 1 ligne par client, avec total_sales_amount (somme des ventes).

2. Clustering K-means

- Vous sélectionnez une seule variable numérique pour le clustering :

```
feature_cols = ["total_sales_amount"]
```

- Vous assemblez cette variable dans une colonne features (VectorAssembler).
- Vous entraînez un modèle K-means avec k=3

Ask questions and work with this notebook

+ Agent

AI-generated content may be incorrect

Selected Cell 8 of 8 cells

KQL queryset

- [Générer des requêtes KQL \(preview\).](#)
- [Modifier ou explorer une requête KQL générée précédemment \(preview\).](#)
- [Générer du KQL pour les raccourcis \(preview\).](#)

Comprendre où Copilot apporte le plus de valeur

Copilot dans Microsoft Fabric ne remplace pas l'expertise data : il **accélère**, **débloque** et **standardise**.

Les usages où Copilot est vraiment pertinent :

Domaine	Apport réel
Data Engineering	Génération de transformations, explication de pipelines
SQL / Lakehouse / Warehouse	Aide à l'écriture, compréhension et refactorisation de requêtes
Data Science	Notebooks Python, feature engineering, explication de modèles
Power BI intégré	Mesures DAX, narration, interprétation
Documentation	Résumés, commentaires, explication des flux

Où il est moins efficace :

- Logique métier complexe implicite
- Cas de performance très spécifiques
- Design d'architecture cible (il aide, mais ne décide pas)

Le « vibe coding », c'est quoi ?



Coder en dialoguant avec l'IA : décrire l'intention, laisser l'assistant proposer, puis itérer. Un gain de vitesse réel — mais qui doit être encadré pour rester fiable.



Bénéfices

- Prototypage et exploration d'API plus rapides
- Moins de code répétitif (*boilerplate*)
- Documentation et tests générés à la demande
- Montée en compétence accélérée



Risques sans cadre

- Dette technique et code peu lisible
- Hallucinations : API ou paramètres inexistants
- Code accepté sans tests ni revue
- Fuite de secrets, dépendances douteuses

Les 3 règles d'or du vibe coding encadré

1

Contexte explicite

Ouvrir les bons fichiers, fournir schémas et conventions. L'IA répond à ce qu'elle voit — pas à ce qu'on imagine.

2

Petites itérations

Valider étape par étape. Jamais 200 lignes acceptées d'un coup : on lit, on teste, on avance.

3

Rien sans test + revue

Aucun code généré n'est mergé sans tests verts et revue humaine en pull request.

Panorama de GitHub Copilot



Complétion inline

Suggestions au fil de la frappe, guidées par le code et les commentaires.



Copilot Chat

Questions / réponses et commandes : /explain, /fix, /tests, /doc.



Copilot Edits

Modifications cohérentes sur plusieurs fichiers à la fois.



Agent mode

Tâches autonomes multi-étapes, sous supervision.



Contexte ciblé

#file, #selection, @workspace pour cadrer la réponse.



copilot-instructions.md

Conventions d'équipe injectées automatiquement dans chaque prompt.

Pourquoi VS Code plutôt que le portail Fabric ?

Portail Fabric (web)

- Git natif limite sur les notebooks
- Debugging restreint
- Pas de tests unitaires
- Refactoring multi-fichiers pénible
- Copilot natif puissant mais cantonné au périmètre sélectionné

VS Code + Copilot

- Git, branches, PR, code review fluides
- Tests pytest ou autres
- Refactoring multi-fichiers
- Extension Fabric Data Engineering : édition locale, exécution sur cluster Spark Fabric
- Copilot voit l'ensemble du repo (fichiers, contrats, exemples)

Exécuter le notebook : local ou distant ?



« Local » = le calcul, pas les données. Les données du Lakehouse vivent dans OneLake (le cloud), jamais sur votre machine.

RECOMMANDÉ POUR VALIDER

Runtime Fabric distant

- Le calcul tourne sur le pool Spark du workspace
- Obligatoire pour notebookutils / mssparkutils
- Comportement fidèle : Delta, V-Order, Lakehouse attaché
- C'est l'environnement de production

POUR ITÉRER VITE

Spark local

- Conda local (runtime 1.1/1.2) ou devcontainer Docker
- Itération rapide : lint, tests, Git, Copilot
- Accès OneLake via chemin ABFSS + Service Principal
- Runtime 1.3+ : plus de conda local ; notebookutils indisponible

Règle : itérer en local, mais toujours valider sur le runtime distant avant de publier.

1

Fondamentaux - installer & configurer

Stack à installer

- VS Code (à jour)
- Extension GitHub Copilot
- Extension GitHub Copilot Chat
- Extension Fabric (Microsoft)
- Extension Fabric Data Engineering (Microsoft)
- Python, Pylance, Jupyter
- Extension Jupyter
- [optionnel] Ruff / Black formatter

Authentification & sécurité

- Compte GitHub d'organisation (pas de compte personnel)
- Activer *content exclusion* sur les repos sensibles
- Connexion à Fabric via l'extension dédiée
- Détection de code public similaire : activée

2

Le contexte fait 80% de la qualite

80%

*de la qualite d'une suggestion vient du contexte
fourni*

Sources de contexte que Copilot utilise

- Le fichier ouvert (curseur + autour)
- Les onglets ouverts dans VS Code
- Le texte selectionné
- Les commentaires en haut du fichier
- Le fichier .github/copilot-instructions.md
- Les exemples présents dans le repo (exemples/)

Règle : si Copilot hallucine, ce n'est pas Copilot qui est mauvais - c'est le contexte qui manque.

3 patterns de prompts efficaces en data engineering

1

Pattern schéma en commentaire

Coller un DESCRIBE TABLE ou la liste des colonnes en commentaire AVANT la fonction. Precision +++ sur les transformations.

2

Pattern contrat YAML vers notebook

Fichier contracts/silver_<table>.yaml decrit les colonnes, types, regles. Demander a Copilot de generer le notebook a partir du contrat + template.

3

Pattern moteur cible explicite

Preciser : genere ceci en PySpark DataFrame API, pas en Spark SQL, cible Fabric Lakehouse Runtime 1.3. Evite les confusions T-SQL / Spark.

Le fichier `.github/copilot-instructions.md`

Lu automatiquement à chaque interaction Copilot Chat. Le levier numéro 1 pour la qualité du code généré.

Contexte projet

Plateforme : Microsoft Fabric (Lakehouse + Warehouse).

Architecture : medallion Bronze/Silver/Gold sur OneLake (Delta).

Langages : PySpark 3.5, Spark SQL, T-SQL (Warehouse).

Conventions

- Notebooks : 1 notebook = 1 table cible. Nommage nb_<layer>_<table>.py
- Tables Delta : snake_case, partition par ingestion_date en Bronze.
- Secrets : notebookutils.credentials, jamais en clair.

A éviter

- Pas de `.collect()` sur DataFrames non bornes
- Pas d'API Synapse legacy (mssparkutils -> notebookutils)
- Pas de code Databricks (dbutils, %run) - Fabric ne le supporte pas

3

Cas d'usage Fabric à forte valeur

Génération medallion

Bronze/Silver/Gold a partir d'un contrat YAML et d'un template du repo.

Tests unitaires PySpark

Cas limites (nulls, doublons, types) generes avec pytest + chispa.

Migration T-SQL vers Spark SQL

/explain + /fix sur les fonctions non supportées (CHARINDEX, STRING_AGG...).

Pipelines fabric-cicd

YAML Azure DevOps / GitHub Actions pour déployer workspaces et items.

Scripts d'admin Fabric

Provisioning workspaces, lakehouses, capacités via Fabric REST API.

Documentation auto

Fiches de tables markdown + schemas JSON à partir des Delta tables.

4

Checklist de revue d'un code généré

Correction

- Gestion des nulls explicite
- Types de colonnes vérifiés
- Pas de `.collect()` cache
- `broadcast()` documenté si table < 50 MB

Securite

- Aucun secret en clair
- `notebookutils.credentials` utilisé
- Pas de chemin OneLake en dur

Gouvernance

- Code relu ligne par ligne
- Pas de copie de code public détectée
- Prompt anonymisé si données sensibles

Prompting efficace pour Data Engineers



Donner le contexte

Schémas, fichiers, conventions. Référencer #file, #selection, @workspace.



Spécifier entrées / sorties

Types, contraintes, volumétrie, idempotence attendue.



Itérer petit à petit

Une intention à la fois ; reformuler plutôt qu'accumuler.



Demander les tests

Systématiquement /tests, avec cas nominaux et cas limites.



Faire justifier le code

« Explique pourquoi cette approche » avant de valider.



Capitaliser

Gabarits de prompts et copilot-instructions.md partagés.

Pièges spécifiques à Fabric que Copilot ignore

Le modèle a été massivement entraîné sur Databricks et Synapse Analytics. À l'équipe de cadrer les spécificités Fabric.

Suggestion Copilot (à corriger)	Bonne pratique Fabric
<code>dbutils.fs / dbutils.secrets</code>	<code>notebookutils.fs / notebookutils.credentials</code>
<code>%run "./other_notebook"</code>	<code>notebookutils.notebook.run('other_notebook')</code>
<code>spark.read.format('delta').load('/mnt/...')</code>	Chemin abfss:// ou nom de table Lakehouse
<code>mssparkutils</code> (Synapse legacy)	<code>notebookutils</code> (Fabric)
<code>CREATE TABLE</code> T-SQL types Synapse	Spark SQL ou T-SQL Fabric Warehouse

A R E T E N I R

3 idées-clés

1

Copilot n'est pas magique : le contexte (instructions, exemples, schema) fait 80% de la qualité.

2

Sur Fabric, le modèle penche vers Databricks et Synapse. C'est à l'équipe de cadrer.

3

Sans champion, sans rituel et sans métrique, la formation s'évapore en 3 mois / semaines / jours.

Sécurité & gouvernance



Aucun secret exposé

Variables d'environnement ou Key Vault — jamais dans le code ni dans les prompts.



Maîtriser les données sensibles

Anonymisation et périmètre clair avant tout partage avec l'assistant.



Revoir les dépendances

Vérifier chaque librairie suggérée : version, maintenance, licence.



Content exclusions & politiques

Fichiers exclus du contexte, politiques d'entreprise Copilot appliquées.



Traçabilité en pull request

Tout code généré passe en PR : revue humaine et historique conservés.



Conformité & responsabilité

L'auteur reste responsable du code mergé — Copilot assiste, ne décide pas.

Les 4 étapes clés et leur logique d'enchaînement



ETAPE 1

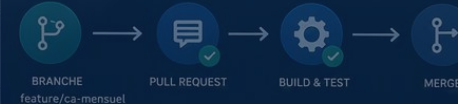
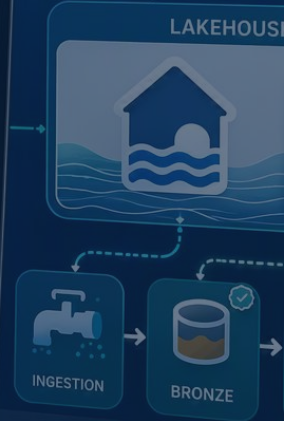
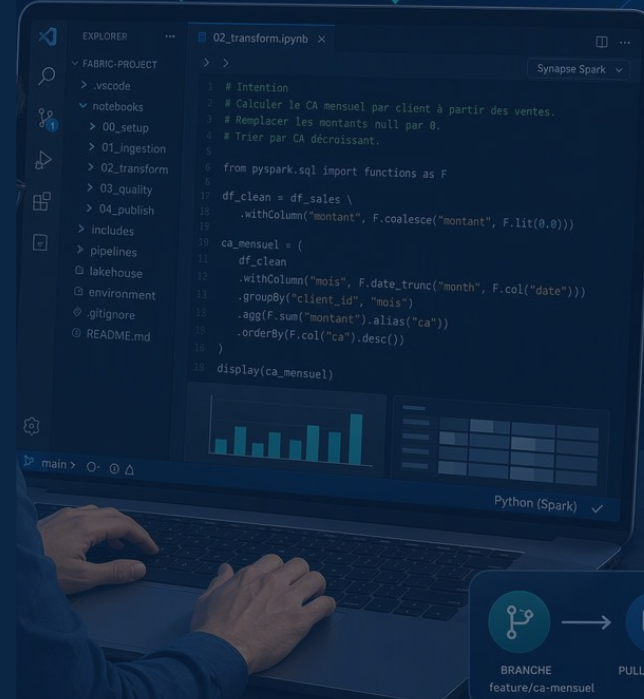
Développer dans VS Code

Local vs distant : distinguer le confort de développement du runtime de production.

du code

RUNTIME LOCAL
Exécution locale

RUNTIME CLOUD
Exécution distante



VS Code comme environnement de développement Fabric

À installer

- Fabric Data Engineering : explorer Lakehouse, développer et déboguer des notebooks Spark
- Microsoft Fabric : gérer workspaces et items
- Jupyter + JDK + conda si exécution locale

Deux modes de travail

Local mode : télécharger l'item, éditer, publier.
VFS mode : éditer les items distants sans téléchargement, avec plusieurs workspaces dans une même fenêtre.

Point clé

Les données du Lakehouse restent dans OneLake. "Local" concerne le moteur de calcul, pas l'emplacement des données.

Runtime Fabric distant	Spark local
Calcul sur le pool Spark du workspace	Conda local ou devcontainer Docker
Obligatoire pour notebookutils / mssparkutils	Itération rapide : lint, tests, Git, Copilot
Comportement fidèle : Delta, V-Order, lakehouse attaché	Accès OneLake via chemin ABFSS + Service Principal
Environnement de production	Runtime 1.3+ : plus de conda local intégré

File

Edit

Selection

View

Go

Run

Terminal

Help

EXPLORER

WORKSPACES [FABRIC DATA ENGINEERING]

> .vscode

> ws_assisted_dev

> Environments

> Lakehouses

> Notebooks / nb_city_safety_ui

> builtin

> dependencies

nb_city_safety_ui.ipynb

ws_assisted_dev

Generate

OUTLINE

TIMELINE

WS Assisted Dev

Select a Kernel from Fabric VS Code Collection

PySpark python3

Python

Spark (Scala)

Spark SQL

Recommended

Jupyter Kernel

Recent Runs

Select Kernel

Pipeline medaillon CITY (Bronze -> Silver -> Gold)

Ce notebook charge le CSV en Bronze (PySpark), puis construit Silver et Gold en Spark SQL avec nommage 4 parties

workspace.lakehouse.schema.table.

Bronze: creation du schema CITY dans le Lakehouse par default (lkh_brz_demo)

spark.sql("CREATE SCHEMA IF NOT EXISTS CITY")

Lecture du CSV bronze (header=true, inferSchema=true)

df = (

spark.read

.option("header", "true")

.option("inferSchema", "true")

[1]

Python

PROBLEMS

OUTPUT

TERMINAL

AZURE

PROMPT FLOW

DEBUG CONSOLE

PS C:\Users\ppeton>

Fabric Data Engineering

0 0

Fabric Data Engineering: ppeton@expertime.com

Cell 1 of 4

FileEditSelectionViewGoRunTerminalHelp

Workspaces [Fabric Data Engineering]

11

EXPLORER

WORKSPACES [FABRIC DATA ENGINEERING]

.vscode

ws_assisted_dev

Environments

Lakehouses

Notebooks / nb_city_safety_ui

builtin

dependencies

nb_city_safety_ui.ipynb

nb_DS_formation_TS.ipynb

nb_city_safety_ui.ipynb

ws_assisted_dev > Notebooks > nb_city_safety_ui > nb_city_safety_ui.ipynb > ...

GenerateCodeMarkdownRun AllRestartClear All OutputsPySpark

Ecriture idempotente en Bronze

(

df.write

.mode("overwrite")

.option("overwriteSchema", "true")

.saveAsTable("CITY.city_safety")

)

print(f"Bronze CITY.city_safety row count: {df.count()}")

print(f"Bronze CITY.city_safety row count: {row_count}")

[2]Python

...

...

Bronze CITY.city_safety row count: 7350

%%sql

CREATE SCHEMA IF NOT EXISTS ws_assisted_dev.lkh_slv_demo.CITY;

CREATE OR REPLACE TABLE ws_assisted_dev.lkh_slv_demo.CITY.city_safety AS

SELECT

dataSubtype,

category,

NULLIF(TRIM(subcategory), 'NULL') AS subcategory,

NULLIF(TRIM(status), 'NULL') AS status,

NULLIF(TRIM(source), 'NULL') AS source,

to_timestamp(dateTime, 'yyyy-MM-dd HH:mm:ss') AS event_datetime,

CAST(latitude AS double) AS latitude,

CAST(longitude AS double) AS longitude,

year(to_timestamp(dateTime, 'yyyy-MM-dd HH:mm:ss')) AS event_year,

month(to_timestamp(dateTime, 'yyyy-MM-dd HH:mm:ss')) AS event_month,

current_timestamp() AS Sid_LoadTimestamp,

'city_safety_seattle.csv' AS Sid_SourceFile

KeepUndo1 of 2Python

CHAT

AJOUTER TEST SUR NOMBRE DE LIGNES DF

ajoute un test sur le nombre de lignes de df

Je vais ouvrir le notebook actif et localiser où df est créé afin d'ajouter un test sur son nombre de lignes au bon endroit, puis je mettrai à jour la cellule concernée.

Je vais d'abord lire la portion sélectionnée du notebook pour insérer le test exactement là où df est manipulé, sans toucher au reste.

Reviewed nb_city_safety_ui.ipynb, received a warning

J'ai trouvé la cellule où df est lu puis écrit en Bronze. Je vais y ajouter un test explicite sur le nombre de lignes (avec échec si 0) et réutiliser ce compteur pour l'affichage final.

Warning from Post-Tool Use hook

Preparing

Working

1 file changed +0 -0KeepUndo

nb_city_safety_ui.ipynb • Cell 2

An expert assistant of Fabric Data Engineering extension that helps u...

FabricNotebookAuto

LocalDefault Approvals

Fabric Data Engineering

000

Fabric Data Engineering: ppeton@experttime.com

Spaces: 4

Cell 2 of 4

ETAPE 2

Versioning avec Git intégration Fabric

Connecter le workspace au dépôt, sérialiser les items et
instaurer le workflow de revue.



Ce que Git versionne dans Fabric — et ce qu'il ne touche pas

Point clé

Seules les métadonnées sont versionnées. Git ne touche jamais aux données des tables Delta ni aux fichiers du Lakehouse.

Se versionne	Ne se versionne pas
Définition des notebooks (code)	Données des tables Delta
Définition des data pipelines	Fichiers stockés dans OneLake
Métadonnées du Lakehouse : nom, GUID	Outputs d'exécution des notebooks
Métadonnées des shortcuts OneLake	État runtime : sessions, caches

Stratégie branches, workspaces et Pull Requests



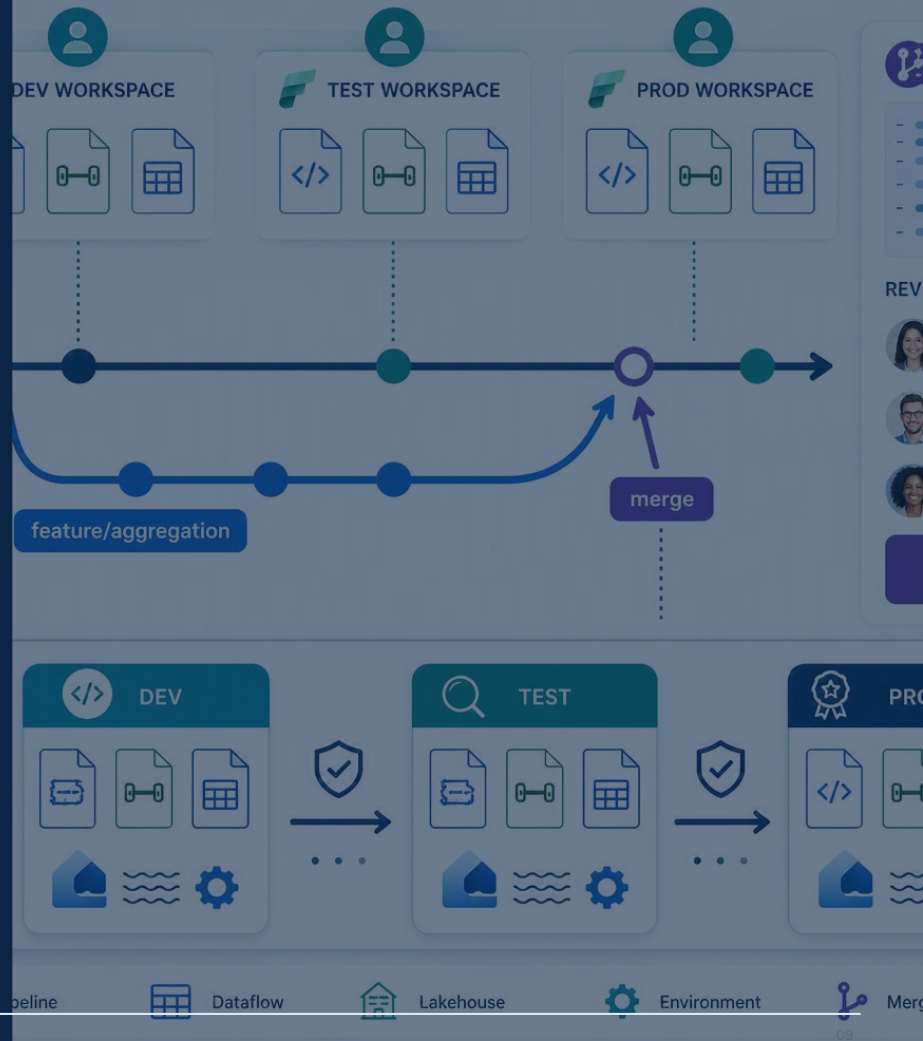
Règles d'équipe

- PR obligatoire vers dev / main
- Pas de commit direct
- Squash merge pour un historique lisible
- Revue ciblée sur le code généré par l'assistant

ETAPE 3

Notebooks propres et versionnables

Réduire les conflits de merge et sécuriser le passage d'un environnement à l'autre.



Checklist d'un notebook prêt à être commité



Vider les outputs avant chaque commit



Nommer les cellules et structurer en sections



Une cellule = une étape logique



Externaliser secrets et connexions



Paramétrer le notebook plutôt que coder les chemins



Relire le diff Git avant de pousser

Pourquoi ?

Un notebook avec outputs et chemins codés en dur produit des diffs illisibles, des conflits de merge et casse au déploiement entre environnements.

Assainir un notebook « sale »

Point de départ

Notebook fourni avec outputs présents, chemin Lakehouse codé en dur et cellules fourre-tout.

Actions à réaliser

Vider les outputs, découper les cellules, remplacer le chemin en dur par un paramètre widget, générer un en-tête de documentation avec l'assistant.

Validation

Commiter et comparer le diff avant / après.

AVANT

Outputs + chemins en dur + cellules monolithiques
→ diff long, bruité, fragile



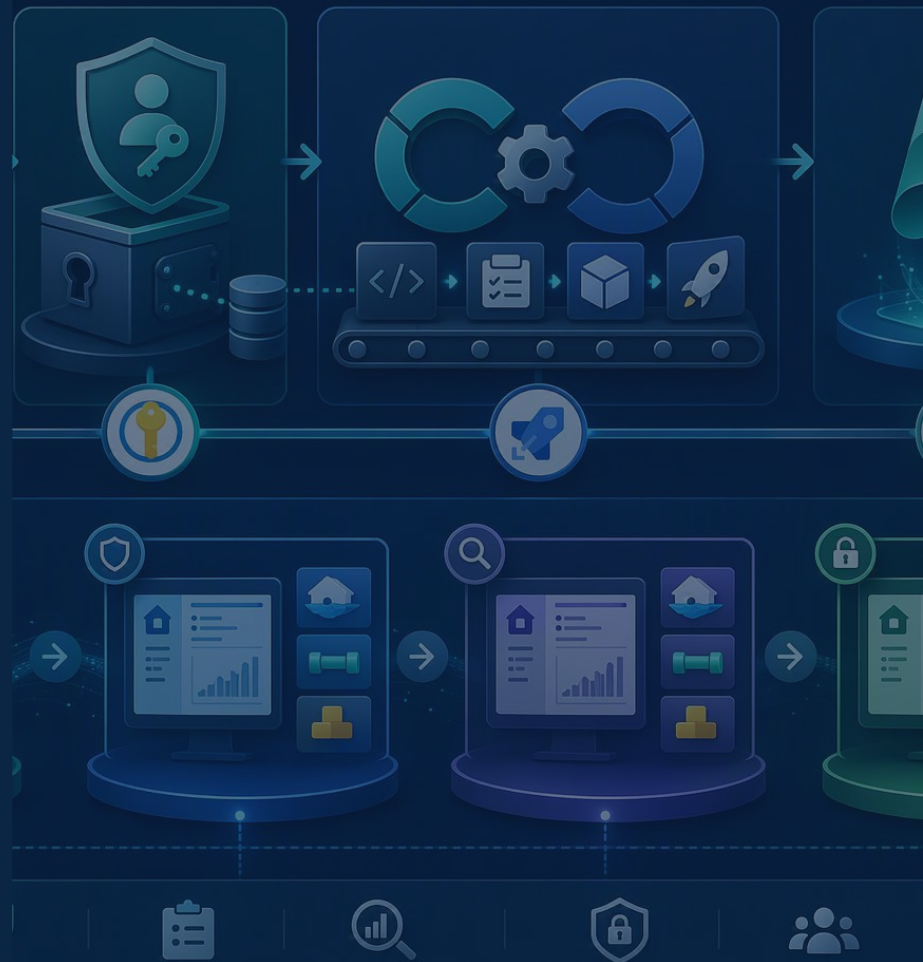
APRÈS

Diff court et lisible, aucun secret, aucun chemin en dur
→ notebook paramétré par environnement

ETAPE 4

CI/CD pilotée par code

Industrialiser les déploiements Fabric avec fabric-cicd, GitHub Actions ou Azure DevOps.



Briques nécessaires pour un déploiement code-first

fabric-cicd

`publish_all_items()` déploie le dépôt vers un workspace cible

Variable Libraries

Paramétrage runtime : Notebooks, Pipelines, Dataflows

parameter.yml

Remplace GUID / connexions par environnement

Service Principal + Key Vault

Authentification sécurisée à l'API Fabric

